Fast $(1, s)$-Nucleus Decomposition via a Static Clique Path Index

Anonymous Authors
Submission #XXX

Abstract

Extracting cohesive substructures from large graphs is a fundamental task in graph mining, with applications in community detection, dense-subgraph search, and role analysis. The $(1, s)$-*nucleus decomposition* is a higher-order generalization of k-core that ranks each vertex by the s-clique witnesses surrounding it (triangles for $s = 3$, 4-cliques for $s = 4$, and so on), giving sharper rankings than the standard k-core but at much higher cost. Existing studies compute $(1, s)$-nucleus inside a general (r, s) framework that encodes the witnesses with a Clique Path Index but maintains the index as mutable residual state during peeling. However, this mutability causes significant overhead in residual-index rewrites, memory consumption, and per-pop work, limiting scalability to large graphs and high s. We observe that the $(1, s)$ special case has a stronger invariant: vertex peeling never deletes edges, so every CPI path remains structurally valid throughout the peel. This invariant turns exact decomposition into counter maintenance over a static CPI, which yields SPIN, a direct static-path iteration, SPIN*, its incremental optimization, and a parallel builder for the shifted construction bottleneck. Experiments on 307 matched successful configurations show that SPIN* achieves a 13.50× geometric-mean peel-time speedup and a 44.76× maximum peel-time speedup over the mutable baseline, while reducing memory usage by a 1.55× geometric-mean ratio; end-to-end speedup is 1.10× because construction becomes the dominant shared cost, which the parallel builder then addresses.

CCS Concepts

• **Theory of computation** → **Graph algorithms analysis**; • **Information systems** → *Data mining*.

Keywords

nucleus decomposition, clique counting, cohesive subgraphs, hierarchy

ACM Reference Format:

Anonymous Authors. 2027. Fast $(1, s)$-Nucleus Decomposition via a Static Clique Path Index. In *Proceedings of 2027 ACM SIGMOD International Conference on Management of Data (SIGMOD '27).* ACM, New York, NY, USA, 16 pages. https://doi.org/XXXXXXX.XXXXXXX

1 Introduction

Core decomposition ranks vertices by the amount of cohesive structure that still surrounds them after weak vertices are removed [1, 12, 14]. The classical k-core uses edges as the witness of cohesion. Many graph mining tasks need a stronger witness, because a group may pass an edge-degree threshold while containing few triangles, 4-cliques, or larger coordinated patterns. The $(1, s)$-*nucleus decomposition* [11] gives this vertex-centric higher-order ranking. For a fixed s, it assigns each vertex the largest value k for which the vertex belongs to a maximal region where every vertex participates in at least k internal s-cliques. At $s = 2$, this is the k-core. At $s = 3$, it ranks vertices by triangle-supported cohesion. At larger s, it exposes progressively tighter clique-based kernels.

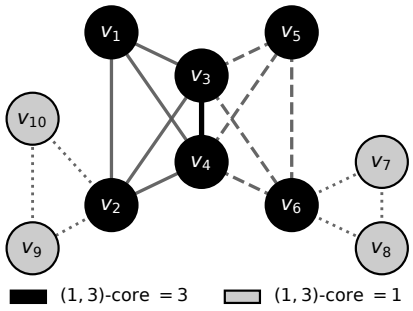

Figure 1: Running example of $(1, 3)$-nucleus decomposition.

Example 1.1. Figure 1 shows a 10-vertex graph at $s = 3$. The six vertices $v_1, \ldots, v_6$ belong to two triangle-rich K_4 blocks; each of them participates in at least three triangles supported by other vertices of the same value, so all six have $(1, 3)$-core value 3. The remaining four vertices $v_7, \ldots, v_{10}$ belong to a single pendant triangle, supporting only one triangle each, so all four have $(1, 3)$-core value 1. A formal definition is given in Section 2; we revisit this graph as a running example throughout the paper.

Challenges. Computing $(1, s)$-nucleus decomposition at high s is hard because the number of s-cliques can be enormous; the current state of the art avoids explicit enumeration with the Clique Path Index, or CPI [9]. A CPI path stores a hold set and a pivot set; it compactly represents all s-cliques formed by taking every hold vertex and choosing the remaining vertices from the pivot set. This representation makes general (r, s)-nucleus decomposition practical, but using it efficiently for the vertex-centric case $r = 1$ raises three challenges.

Residual state during peeling. Each peel step must update the surviving support of every remaining vertex. The general (r, s) algorithm achieves this by maintaining the CPI as mutable residual state: it stores per-path vertex sets, a reverse vertex-to-path index, and a priority queue, and rewrites these structures every time a vertex is popped. The challenge is whether this mutability is necessary at $r = 1$, where peeling removes vertices but does not delete edges. If not, residual maintenance becomes pure arithmetic over a static incidence layout.

SIGMOD '27, June 2027, TBD

ACM ISBN 978-1-XXXX-XXXX-X/27/06…$15.00
https://doi.org/XXXXXXX.XXXXXXX

Memory footprint of the residual index. The residual structures that support peeling also dominate memory. Per-path vertex hash sets, the reverse vertex-to-path map, and the live Bron–Kerbosch recursion tree together pay constant-factor overhead per vertex–path incidence on top of the clique-encoding cost. The challenge is to encode the same s-clique witnesses with a layout whose memory grows with the vertex–path incidence count Σ alone, so that the achievable s on a fixed-memory machine is dictated by Σ rather than by hash-table overhead.

Construction as the new bottleneck. Once peeling becomes counter-only, the cost profile shifts: on many real graphs the static index takes seconds to minutes to construct while peeling takes milliseconds, so the index builder is the next bottleneck. The challenge is to construct the static CPI in parallel without paying the synchronisation cost that the mutable design requires for path rewrites.

State-of-the-Art and Limitations. The current state of the art for general (r, s)-nucleus decomposition is CND [9], the mutable-CPI algorithm. CND addresses the general case correctly, but inherits three weaknesses when restricted to $r = 1$.

Mutable residual-index maintenance. CND rewrites path records and updates a hash-indexed reverse map at every peel step. This work is intrinsic to the mutable design. Our ablation on 171 matched cells shows that simply replacing the mutable index with a static one (still using a full-pass solver) already gives a 3.80× geometric-mean peel-time speedup; the residual rewrites are a real cost, not a constant factor.

High memory consumption. The residual structures that CND must keep mutable in the general (r, s) setting — per-path membership queries, reverse vertex-to-path lookup, and the live recursion tree used to rewrite paths — together cost roughly 88Σ bytes per incidence (Section 8, Table 4), close to an order of magnitude more than what a static encoding of the same witnesses requires. The geometric-mean memory ratio CND/SPIN* is 1.55× on the matched cells and 3.92× at maximum; on `com-friendster` (the largest SNAP graph) the overhead pushes CND past the 503 GB available at every s, so the algorithm never completes on that graph.

Construction as the residual bottleneck. Even after cheap peeling, CND pays the full construction cost; on `web-it-2004` at $s = 3$, construction takes about 127 seconds and peeling takes 351 milliseconds, so the static index builder dominates total runtime by 360× on this configuration. Without a parallel builder, the end-to-end speedup over CND stays at 1.10× on the matched cells even though the peel-time speedup is 13.50×.

Our Solution and Contributions. For $(1, s)$-nucleus decomposition we show that vertex peeling preserves the CPI structure: a set of vertices that formed a clique when the CPI was built remains a clique throughout the peel. A path stops contributing only when one of its hold vertices is removed; otherwise its only dynamic state is an integer counter of how many of its pivots remain. This invariant turns exact decomposition into counter arithmetic over a static incidence layout, which in turn unlocks an incremental peel schedule and a parallel builder. Our contributions address the three challenges one-for-one and add two further pieces that follow from the same static-CPI foundation.

*Static-*CPI *invariant for* $(1, s)$ *(solves Limitation 1).* We prove that vertex peeling never rewrites the stored hold or pivot sets (Theorem 4.1) and give a closed-form expression for the support loss caused by each path event (Lemma 4.1). The dynamic state of a path collapses to a liveness bit and a single integer counter, so the per-peel residual rewrites disappear and the ablation in RQ7 confirms a 3.80× geometric-mean peel-time speedup over CND from this change alone.

Elimination of residual structures (solves Limitation 2). A corollary of Theorem 4.1 is that three of CND's residual structures cease to carry information once the invariant is established: (i) the per-path vertex hash sets, because membership in a path is already determined by the path's fixed hold/pivot sets; (ii) the reverse vertex-to-path map, because the vertex-to-path relation is fixed at construction and never edited; and (iii) the mutable Bron–Kerbosch recursion tree, because no path is ever rewritten during peeling. This collapses the per-incidence space cost from $\approx 88\Sigma$ bytes to $\approx 10\Sigma$ bytes (Table 4); the measured geometric-mean memory ratio CND/SPIN* is 1.55× over 307 matched cells, and `com-friendster` (the largest SNAP graph) becomes decomposable for $s \in \{2, \ldots, 6, 11\}$ on a 503 GB machine on which CND cannot start.

*Parallel static-*CPI *construction (solves Limitation 3).* Because the static-CPI output is invariant under the order in which seed vertices are processed and contains no mutable shared state, seed enumeration partitions into independent subproblems. ParaBuild (Algorithm 4) constructs the index by running these subproblems in parallel and combining their outputs by a deterministic merge whose result does not depend on the schedule; the empirical scaling reaches 23.4× at T=24 threads.

The three contributions above address the three limitations one-for-one. Two further contributions follow from the same static-CPI foundation:

Direct and incremental solvers over the static index. We formulate SPIN as the direct threshold-support iteration on CPI witness counts (Algorithm 2) and SPIN* as its incremental optimization (Algorithm 3) that computes the same fixed point in peel order without repeated full-graph passes (Lemma 5.3). SPIN* delivers a 36.2× geometric-mean peel-time speedup over SPIN on 291 matched cells.

Core-value hierarchy from the static CPI. After peeling, each CPI leaf can be treated as a hyperedge born at the minimum core value among its stored vertices. A descending union–find scan over the leaves recovers the s-clique-connected super-level join tree in $O(\Sigma \log \Sigma)$ time without re-enumerating s-cliques (Algorithm 5); the hierarchy is a byproduct of preserving the CPI rather than a separate decomposition problem.

Comprehensive experimental evaluation. We compare against CND on ten real-world graphs (307 matched configurations, headline peel speedup 13.50× gmean / 44.76× max, memory-usage ratio 1.55× gmean), report phase behaviour and parallel construction, push the pipeline to a 1.8B-edge graph (`com-friendster`, where CND cannot complete), and use case studies to evaluate whether the $r = 1$ specialization is practically useful.

The rest of the paper follows this dependency chain. Section 2 defines the problem, the CPI, and the mutable baseline. Section 4 derives the static-CPI invariant from the vertex-removal rule. Section 5 derives SPIN and SPIN* from the same threshold-support fixed point. Section 6 parallelizes construction after peeling becomes cheap. Section 7 extracts the hierarchy from static CPI leaves

after peeling. Sections 8 and 9 report performance and application evidence. Sections 10 and 11 position and conclude the paper.

2 Preliminaries

The introduction stated the paper's central invariant in informal terms. To prove and use it, we need precise objects: vertex support, $(1,s)$-nucleus core values, CPI paths, and the mutable baseline that the paper specializes. This section fixes that vocabulary and ends at the point where the $r = 1$ simplification can be stated cleanly.

Graph notation. We work with a simple undirected graph $G = (V, E)$. For a vertex v, $N_G(v) = \{u \in V \mid (u, v) \in E\}$ is its neighbor set and $\deg_G(v) = |N_G(v)|$ is its degree. For $X \subseteq V$, $G[X]$ is the subgraph induced by X. The implementation stores G in CSR form with an `offsets` array and an `edges` array.

Definition 2.1 (k-Clique)**.** A *k-clique* is a vertex set $C \subseteq V$ with $|C| = k$ such that every two distinct vertices in C are adjacent.

Definition 2.2 (s-Clique Support)**.** Given $s \geq 2$, the *s-clique support* of a vertex v in G is

$$\deg^{(s)}(v) \;=\; \big|\{S \subseteq V \mid v \in S,\ |S| = s,\ S \text{ is a clique in } G\}\big|.$$

Support is always evaluated inside the current induced subgraph when a peel level is being defined. When the graph is clear, we write $\deg^{(s)}(v)$ without a subscript.

Definition 2.3 (k-$(1,s)$-Nucleus)**.** Let $X \subseteq V$. The induced subgraph $G[X]$ is a *k-$(1,s)$-nucleus* if:

(1) every vertex in X belongs to at least k s-cliques contained in $G[X]$;
(2) $G[X]$ is *s-connected*: for any two vertices $u, v \in X$, there is a sequence $u = v_1, \ldots, v_t = v$ such that each pair $\{v_i, v_{i+1}\}$ is contained in an s-clique of $G[X]$;
(3) $G[X]$ is maximal with respect to adding vertices while preserving the first two conditions.

Thus a k-$(1,s)$-nucleus is a maximal region in which every vertex has at least k internal s-clique witnesses and the region is connected through those witnesses.

Definition 2.4 (Core Number)**.** Given $s \geq 2$, the *$(1,s)$-nucleus core number* of a vertex v, denoted $\kappa_s(v)$, is the largest k for which v belongs to some k-$(1,s)$-nucleus of G.

When $s = 2$, κ_s is the classical core number. When $s = 3$, it ranks vertices by triangle-supported cohesion.

The Clique Path Index. The Clique Path Index, or CPI, compactly represents s-cliques without listing them one by one [9].

Definition 2.5 (CPI Path)**.** A CPI *path* is a pair $\mathcal{P} = (V_h(\mathcal{P}), V_p(\mathcal{P}))$ of disjoint vertex sets such that $V_h(\mathcal{P}) \cup V_p(\mathcal{P})$ is a clique in G and $|V_h(\mathcal{P})| \leq s \leq |V_h(\mathcal{P})| + |V_p(\mathcal{P})|$. The set $V_h(\mathcal{P})$ is the *hold set*: every encoded s-clique contains all of these vertices. The set $V_p(\mathcal{P})$ is the *pivot set*: an encoded s-clique chooses vertices from this pool. The *pivot requirement* of $\mathcal{P}$ is $\eta_{\mathcal{P}} = s - |V_h(\mathcal{P})|$, the number of pivot vertices needed to complete the hold set into an s-clique. The set of s-cliques encoded by $\mathcal{P}$ is

$$C(\mathcal{P}) = \big\{V_h(\mathcal{P}) \cup P' \mid P' \subseteq V_p(\mathcal{P}),\ |P'| = \eta_{\mathcal{P}}\big\}.$$

Its cardinality is $\binom{|V_p(\mathcal{P})|}{\eta_{\mathcal{P}}}$.

The CPI is a set $\mathcal{P}$ of paths such that every s-clique is encoded by exactly one path. We write

$$\Sigma = \sum_{\mathcal{P} \in \mathcal{P}} \big(|V_h(\mathcal{P})| + |V_p(\mathcal{P})|\big)$$

for the vertex–path incidence count. We also write $\deg_\Sigma(v) = |\{\mathcal{P} \mid v \in V_h(\mathcal{P}) \cup V_p(\mathcal{P})\}|$, $d_{\max} = \max_v \deg_\Sigma(v)$, and $\kappa_{\max} = \max_v \deg^{(s)}(v)$. The CPI is built by a degeneracy-ordered Bron–Kerbosch search with Tomita pivoting [2, 4, 9, 13].

Lemma 2.1 (Support Formula [9])**.** *For every $v \in V$,*

$$\deg^{(s)}(v) = \sum_{\mathcal{P}\,:\, v \in V_h(\mathcal{P})} \binom{|V_p(\mathcal{P})|}{\eta_{\mathcal{P}}} + \sum_{\mathcal{P}\,:\, v \in V_p(\mathcal{P})} \binom{|V_p(\mathcal{P})| - 1}{\eta_{\mathcal{P}} - 1}.$$

Problem statement. Given G and $s \geq 2$, compute $\kappa_s(v)$ for every $v \in V$. Section 7 treats hierarchy construction after the core values have been computed. Section 3 describes the mutable-CPI baseline that this paper compares against; the remaining sections specialise it for $r = 1$.

3 The Mutable-CPI Baseline

In this section we describe CND [9], the state-of-the-art exact algorithm for general (r,s)-nucleus decomposition. We first walk through its peeling loop, then enumerate the residual structures it maintains, and finally pinpoint the operations that this paper specialises away when restricted to $r = 1$.

The peeling loop. CND follows the standard core-decomposition template [1]: it maintains the residual support of every alive r-clique in a priority queue, repeatedly pops the r-clique of minimum residual support, assigns it the current peel level as its core value, and propagates the support loss to the remaining alive r-cliques. At $r = 1$ the queue items are vertices, so each iteration pops a vertex v, records $\kappa_s(v)$, and decrements the residual support of every vertex that loses an s-clique witness because of v's removal. The non-trivial work happens during this support-loss propagation, because the CPI that encodes the witnesses must be brought into a state consistent with v being gone before the next pop.

Residual state. To keep the witness counts consistent across peels, CND maintains the CPI as mutable residual state. For each path $\mathcal{P}$ it stores the hold and pivot vertex sets as hash sets so that the test "does v belong to this path?" is constant time. A separate reverse map sends every vertex u to the list of paths that contain it, so that when u is popped the algorithm can find the affected paths without scanning all of $\mathcal{P}$. A heap (or bucket queue) keeps every alive vertex ordered by current residual support, with decrease-key operations for the support drops triggered at each peel event. Finally, the Bron–Kerbosch recursion tree that produced the CPI is kept mutable: when a peel event invalidates a path, the algorithm walks into the tree and rewrites or removes the affected nodes.

Per-peel work. A single peel of vertex v costs:

- a reverse-map lookup of v's path list, then for every such path $\mathcal{P}$ a constant-time hash-set query to determine whether v is a hold or a pivot;
- a path rewrite: if v is a hold, the path is deleted from the recursion tree and its reverse-map entries for all other vertices in $V_h(\mathcal{P}) \cup V_p(\mathcal{P})$ are erased; if v is a pivot, the path's pivot set shrinks by one and the reverse-map entry for v on this path is erased;

- a support recompute for every path-mate u using a freshly enumerated set of s-cliques that survive the rewrite, followed by a decrease-key on u in the heap.

The dominant components are the path rewrites and the reverse-map erases; together they scale with the number of vertex–path incidences touched by the event, which is what gives CND its observed cost profile.

Why this state is needed in the general case. The mutable design is not over-engineering for the general (r, s) setting. When the peeled object is an edge ($r = 2$) or a higher-order clique ($r \geq 3$), the removal can destroy edges among vertices that remain alive; some of the cliques that a path encodes therefore cease to exist as cliques. In that setting, the stored hold and pivot sets of $\mathcal{P}$ no longer define a valid encoded clique family, so the path must be split, shrunk, or deleted by walking back into the recursion tree. The reverse map and the per-path hash sets exist precisely to support that surgery efficiently.

Why this state is wasteful at $r = 1$. The $(1, s)$ case has a stronger invariant: vertex peeling removes vertices but does not delete edges, so a set of vertices that formed a clique when the CPI was built remains a clique throughout the peel (we prove this in Section 4). A path's encoded clique family therefore stays structurally valid; only the question of whether a particular vertex of $V_h(\mathcal{P}) \cup V_p(\mathcal{P})$ is still alive changes. This means the path rewrites, the reverse-map erases, and the recursion-tree surgery in the per-peel cost above are all unnecessary at $r = 1$ — they maintain a mutable residual state that is logically equivalent to a static incidence layout plus a single integer counter per path. The next four sections cash in this observation: Section 4 states the static-CPI invariant in counter form; Section 5 derives SPIN and SPIN* over the resulting static index; Section 6 parallelises construction once peeling becomes counter-only; and Section 7 extracts the nucleus hierarchy from the preserved CPI without any clique re-enumeration.

4 Static CPI for $r = 1$

In this section, we prove the central invariant of the paper: at $r = 1$, the residual state that CND maintains (Section 3) is unnecessary. Vertex peeling removes vertices from the active subgraph, but it does not delete edges among the vertices that remain. Therefore, the stored clique $V_h(\mathcal{P}) \cup V_p(\mathcal{P})$ of every path remains structurally valid for as long as its active vertices are considered. Only the contribution of the path changes.

Static structure and dynamic counters. A path stores a fixed recipe for its encoded s-cliques. The hold set $V_h(\mathcal{P})$ is the part that every encoded clique must contain. The pivot set $V_p(\mathcal{P})$ is the pool from which the path chooses the remaining vertices. The pivot requirement $\eta_{\mathcal{P}} = s - |V_h(\mathcal{P})|$ says how many still-effective pivots must be chosen from that pool. These three objects are static. During peeling, the algorithm does not edit the stored hold or pivot sets. The dynamic state is only a liveness bit and an integer $p_{\mathcal{P}}$, the number of pivots in $V_p(\mathcal{P})$ that have not yet been peeled. If a peeled vertex is a hold, every encoded s-clique contains that vertex, so the path contributes nothing after the event. If a peeled vertex is a pivot, the path remains the same symbolic object, but $p_{\mathcal{P}}$ decreases by one. This is the whole state transition. The next theorem gives the counter form we need: a peel event either kills a path contribution or reduces an effective pivot count.

Theorem 4.1 (Vertex Removal on CPI, Counter Form)**.** *Let $\mathcal{P}$ be a path of the* CPI *of G, and let $v \in V$ be removed from G.*

(1) If $v \in V_h(\mathcal{P})$, $\mathcal{P}$ encodes no surviving s-clique.

(2) If $v \in V_p(\mathcal{P})$, the surviving s-cliques are encoded by the same $V_h(\mathcal{P})$ together with $V_p(\mathcal{P}) \setminus \{v\}$. Their count depends only on $|V_p(\mathcal{P})|$. A counter $p_{\mathcal{P}} = |V_p(\mathcal{P})|$ decremented by one is therefore support-equivalent to deleting v from $V_p(\mathcal{P})$. If $p_{\mathcal{P}} - 1 < \eta_{\mathcal{P}}$, no surviving s-clique is encoded.

(3) Otherwise, $\mathcal{P}$ is unaffected.

The dynamic state of $\mathcal{P}$ during peeling reduces to an integer counter $p_{\mathcal{P}}$, the fixed pivot requirement $\eta_{\mathcal{P}}$, and a liveness bit. The stored sets $V_h(\mathcal{P})$ and $V_p(\mathcal{P})$ are never modified.

Proof. $\mathcal{P}$ encodes $C(\mathcal{P}) = \{V_h(\mathcal{P}) \cup P' \mid P' \subseteq V_p(\mathcal{P}), |P'| = \eta_{\mathcal{P}}\}$. If $v \in V_h(\mathcal{P})$, every encoded s-clique contains v, so no encoded clique survives after v is removed. If $v \in V_p(\mathcal{P})$, the surviving encoded cliques are exactly

$$\{V_h(\mathcal{P}) \cup P' \mid P' \subseteq V_p(\mathcal{P}) \setminus \{v\}, |P'| = \eta_{\mathcal{P}}\}.$$

This family has cardinality $\binom{|V_p(\mathcal{P})|-1}{\eta_{\mathcal{P}}}$. By Lemma 2.1, every support contribution of the path depends on the pivot set only through this cardinality and the corresponding pivot cardinalities. Thus replacing the physical deletion from $V_p(\mathcal{P})$ by the counter update $p_{\mathcal{P}} \leftarrow p_{\mathcal{P}} - 1$ gives the same support arithmetic. The family is empty exactly when fewer than $\eta_{\mathcal{P}}$ effective pivots remain. If $v \notin V_h(\mathcal{P}) \cup V_p(\mathcal{P})$, no encoded clique on $\mathcal{P}$ contains v, so the encoded family is unchanged. □ □

Boundary of the invariant. The theorem is specific to $r = 1$. When $r \geq 2$, a peel event removes an edge or a higher-order clique from the residual object. That event can break the clique represented by $V_h(\mathcal{P}) \cup V_p(\mathcal{P})$. In that setting, a path may need to be split, shrunk, or rebuilt. The mutable CPI machinery of [9] remains the appropriate general tool, and this paper does not subsume it.

Support loss from one path event. Theorem 4.1 leaves only cardinality changes. The support lost by any other live vertex on the path is therefore a binomial difference.

Lemma 4.1 (Support Loss for a Path Event)**.** *Let $\mathcal{P}$ be a live path with $p \geq \eta$ before an event that removes $d \geq 1$ live pivots from $\mathcal{P}$. For every other alive vertex u incident to $\mathcal{P}$:*

$$\Delta_{hold}(p, \eta, d) = \binom{p}{\eta} - \binom{p-d}{\eta} \qquad \text{if } u \in V_h(\mathcal{P}),$$

$$\Delta_{pivot}(p, \eta, d) = \binom{p-1}{\eta-1} - \binom{p-d-1}{\eta-1} \qquad \text{if } u \in V_p(\mathcal{P}).$$

A path-killing event, caused by a hold peel or by $p - d < \eta$, is the limiting case in which the entire contribution vanishes: $\binom{p}{\eta}$ for surviving holds and $\binom{p-1}{\eta-1}$ for surviving pivots.

Proof. Apply Lemma 2.1 before and after the event. A hold u contributes $\binom{p}{\eta}$ before the event and $\binom{p-d}{\eta}$ after the event. A pivot u occupies one pivot slot. It contributes $\binom{p-1}{\eta-1}$ before the event and $\binom{p-d-1}{\eta-1}$ after the event. Subtracting the after value from the before value gives the two formulas. □ □

Running example. Figure 1 shows the running example at $s = 3$: two K_4 blocks share edge (v_3, v_4), and two pendant triangles attach at v_2 and v_6. The six CPI paths in Table 1 encode the ten

Table 1: CPI paths for the running example at $s = 3$.

Path	V_h	V_p	$\eta = s - \lvert V_h\rvert$	$\binom{\lvert V_p\rvert}{\eta}$
$\mathcal{P}_1$	$\{v_7\}$	$\{v_6, v_8\}$	2	1
$\mathcal{P}_2$	$\{v_9\}$	$\{v_2, v_{10}\}$	2	1
$\mathcal{P}_3$	$\{v_1\}$	$\{v_2, v_3, v_4\}$	2	3
$\mathcal{P}_4$	$\{v_2\}$	$\{v_3, v_4\}$	2	1
$\mathcal{P}_5$	$\{v_3\}$	$\{v_4, v_5, v_6\}$	2	3
$\mathcal{P}_6$	$\{v_4\}$	$\{v_5, v_6\}$	2	1
Total				**10**

triangle witnesses through stored V_h, V_p sets and the binomial count $\binom{|V_p|}{s-|V_h|}$. The $(1,3)$-core values fall into two bands: $\kappa_3(v_i) = 3$ for $i \in \{1,\dots,6\}$ (the two K_4 blocks) and $\kappa_3(v_i) = 1$ for $i \in \{7,\dots,10\}$ (the pendant triangles); this is the partition shown by the node shading in Figure 1. The example exercises both events of Theorem 4.1: a path dies when a hold vertex is removed, and a path's pivot counter shrinks when a pivot vertex is removed.

Example 4.1 (Peel event walk-through). Consider the path $\mathcal{P}_3 = (\{v_1\}, \{v_2, v_3, v_4\})$ in Table 1, with $p = 3$ and $\eta = 2$. Peeling v_2 as a pivot triggers Lemma 4.1 with $d = 1$. The surviving hold v_1 loses $\binom{3}{2} - \binom{2}{2} = 2$ from its support. The surviving pivots v_3 and v_4 each lose $\binom{2}{1} - \binom{1}{1} = 1$. The stored set $V_p(\mathcal{P}_3)$ is not modified. Only $p_{\mathcal{P}_3}$ changes from 3 to 2.

5 Computing Core Values over Static CPI

In this section, we turn the static-CPI invariant of Section 4 into a concrete algorithm for computing κ_s. We first derive a one-vertex update rule from clique-witness counts (Section 5.1, Algorithm 1), then derive two solvers that share the same update: SPIN applies it by repeated full passes (Section 5.2, Algorithm 2), while SPIN* schedules it in peel order so unaffected vertices are not revisited (Section 5.3, Algorithm 3). Lemma 5.3 proves that the two solvers compute the same fixed point.

5.1 Vertex updates from static witnesses

The static-CPI invariant tells us how to count witnesses without rebuilding residual paths. To compute core values, we also need a rule for lowering a vertex value from the witnesses that remain. The rule asks for the largest support level that the queried vertex can still justify. At level k, the vertex needs at least k encoded clique witnesses whose other vertices can also sustain level k. For a tentative value vector, define the one-vertex update

$$\Phi_h(v) = \max\{k \geq 0 \mid W_v(h,k) \geq k\}.$$

Here $W_v(h,k)$ is the number of encoded s-clique witnesses through v whose other vertices have value at least k. For clique size two, the witnesses are edges, so the update becomes the standard core fixed-point test over neighbors [5, 7, 8]. For larger clique sizes, the counted items are clique witnesses, not distinct neighbor vertices. The next lemma counts $W_v(h,k)$ from CPI paths without enumerating those witnesses.

Lemma 5.1 (Per-path Witness Count)**.** *Fix a vertex v, a current vector h, and a threshold $k \geq 1$. For a path $\mathcal{P}$ with $v \in V_h(\mathcal{P}) \cup V_p(\mathcal{P})$, let $H_k(\mathcal{P})$ and $Q_k(\mathcal{P})$ be the number of vertices in $V_h(\mathcal{P}) \setminus \{v\}$ and $V_p(\mathcal{P}) \setminus \{v\}$, respectively, whose current h-value is at least k. The number $W_v(\mathcal{P},h,k)$ of encoded s-cliques through v in which every other vertex has h-value at least k is:*

(1) if $H_k(\mathcal{P}) < |V_h(\mathcal{P}) \setminus \{v\}|$, then $W_v(\mathcal{P},h,k) = 0$;

(2) if $v \in V_h(\mathcal{P})$, then $W_v(\mathcal{P},h,k) = \binom{Q_k(\mathcal{P})}{\eta_{\mathcal{P}}}$;

(3) if $v \in V_p(\mathcal{P})$, then $W_v(\mathcal{P},h,k) = \binom{Q_k(\mathcal{P})}{\eta_{\mathcal{P}}-1}$.

The total threshold count is $W_v(h,k) = \sum_{\mathcal{P} \ni v} W_v(\mathcal{P},h,k)$. $\Phi_h(v)$ is the largest k such that $W_v(h,k) \geq k$.

Algorithm 1: VertexSupport(v,h) — one-vertex update from static clique witnesses.

Input: Vertex $v \in V$; value array h with $h[u]$ the running estimate of $\kappa_s(u)$ for each $u \in V$; CPI $\mathcal{P}$.
Output: The threshold-support value $\Phi_h(v)$.

```
1 C[0, ..., h[v]] ← 0 ;                          // threshold witness counts
2 foreach path P ∈ P through v do
3     for k ← 1 to h[v] do
4         C[k] ← C[k] + W_v(P, h, k);                    // Lemma 5.1
5 for k ← h[v] to 1 do
6     if C[k] ≥ k then return k;
7 return 0
```

Proof. Each encoded s-clique through v has the form $V_h(\mathcal{P}) \cup P'$, where $P' \subseteq V_p(\mathcal{P})$ and $|P'| = \eta_{\mathcal{P}}$. If any other hold vertex fails the threshold k, no encoded clique through v can qualify. If v is a hold, all $\eta_{\mathcal{P}}$ pivot slots must be filled by threshold-passing pivots. If v is a pivot, v occupies one pivot slot and the remaining $\eta_{\mathcal{P}} - 1$ slots must be filled by threshold-passing pivots. Summing the qualifying counts over paths gives $W_v(h,k)$. □ □

VertexSupport(v,h) computes this update for one vertex. During one call, the algorithm builds only local threshold state for v. After any prefix of the paths through v, that local state contains exactly the witnesses contributed by the scanned paths. Scanning the next path adds the binomial counts from Lemma 5.1, and the largest threshold with enough witnesses is the updated value. Because the call reads the static CPI but never mutates a path, the same witness rule can be used by both solvers below.

Algorithm 1 zeroes the local threshold-count array up to $h[v]$ (Line 1), accumulates the per-threshold witness count $W_v(\mathcal{P},h,k)$ into $C[k]$ for every path through v using the closed form of Lemma 5.1 (Lines 2-4), and returns the largest k at which $C[k] \geq k$ by scanning C from $h[v]$ downward (Lines 5-7); otherwise it returns 0. A direct implementation does not enumerate clique witnesses: it evaluates the same threshold counts from the values stored on each scanned path, using sorting or buckets, so one call costs $O(\deg_\Sigma(v) \log \deg_\Sigma(v))$. The call reads the static CPI and the vector h, and it mutates no path.

5.2 Direct Static-Path Iteration

VertexSupport defines the threshold-support equation for one vertex. SPIN is the most literal solver: it scans all vertices, computes a next value for each vertex from the previous value vector, and repeats until a full scan makes no change. Its purpose is to expose the fixed-point computation directly. The only mutable data are the current value vector and the next value vector. The current vector starts at the initial s-clique support, so every entry is an upper bound on the final core value. Each pass can only lower entries. A fixed point means every vertex value is consistent with the witnesses whose other vertices have high enough current value.

Algorithm 2: SPIN — direct static-path fixed-point iteration.

Input: Graph G, clique size s, CPI $\mathcal{P}$.
Output: $\kappa_s(v)$ for every $v \in V$.

1 **foreach** $v \in V$ **do**
2 $h[v] \leftarrow \deg^{(s)}(v)$; // closed form from Lemma 2.1
3 changed $\leftarrow$ true;
4 **while** changed **do**
5 changed $\leftarrow$ false; $h_{\text{new}} \leftarrow h$;
6 **foreach** $v \in V$ in a fixed order **do**
7 $h_{\text{new}}[v] \leftarrow \text{VertexSupport}(v, h)$;
8 **if** $h_{\text{new}}[v] \neq h[v]$ **then** changed $\leftarrow$ true;
9 $h \leftarrow h_{\text{new}}$;
10 **foreach** $v \in V$ **do**
11 $\kappa_s(v) \leftarrow h[v]$;
12 **return** κ_s

This makes SPIN the reference formulation for the static-CPI computation, even though repeated full scans can be expensive.

Algorithm 2 initialises $h[v]$ to the initial s-clique support $\deg^{(s)}(v)$ for every vertex via the closed form of Lemma 2.1 (Lines 1-2). The main loop (Lines 3-8) iterates until no entry changes in a full pass: it sets a *changed* flag to false (Line 5), copies the current vector into h_{new} (Line 5), then calls VertexSupport(v, h) for every vertex in a fixed order and stores the result into $h_{\text{new}}[v]$ (Lines 6-7); if any update lowered an entry the flag is set, otherwise the pass exits. The swap $h \leftarrow h_{\text{new}}$ at the end of the pass (Line 8) avoids order dependence inside the pass. The final core values are read off the converged vector (Lines 9-10) and returned (Line 11). A pass with no changed entry is the certificate that the fixed point has been reached.

Example 5.1 (Static-path update on the running example). Initialise $h[v] \leftarrow \deg^{(s)}(v)$ using Lemma 2.1; on the running example $h[v_4] = 7$ (contributions from $\mathcal{P}_3, \mathcal{P}_4$ as pivot, $\mathcal{P}_5, \mathcal{P}_6$ as hold; see Table 1). The first call VertexSupport(v_4, h) scans the four paths through v_4 and counts threshold witnesses via Lemma 5.1. At threshold $k = 3$, paths $\mathcal{P}_3$ and $\mathcal{P}_5$ together contribute $W_{v_4}(h, 3) = 3$ witnesses with all other path vertices at $h \geq 3$, so the largest k with $W_{v_4}(h, k) \geq k$ is $\Phi_h(v_4) = 3$, the final $\kappa_3(v_4)$. SPIN reaches the fixed point in a small number of full passes, but each pass scans every vertex regardless of whether its value can still change; this is the redundancy that SPIN* eliminates in Section 5.3.

The proof below connects the fixed point to $(1, s)$-nucleus core values. It is the reason SPIN can serve as the semantic reference for the incremental algorithm of Section 5.3.

Lemma 5.2 (Fixed Point Correctness). *Initialised at $h^{(0)}[v] = \deg^{(s)}(v)$, Algorithm 2 converges in at most $1 + n\kappa_{\max}$ passes to $h[v] = \kappa_s(v)$ for every $v \in V$. The sequence $h^{(t)}[v]$ is monotonically non-increasing.*

Proof. At any pass, $h^{(t+1)}[v] \geq k$ exactly when at least k incident s-cliques have all other vertices with $h^{(t)} \geq k$. The initial value $\deg^{(s)}(v)$ is the largest possible support value for v, so the update cannot increase $h[v]$. The values are non-negative integers bounded by $\kappa_{\max}$, so the sequence converges. At a fixed point $h^\star$, the set $\{v \mid h^\star[v] \geq k\}$ is exactly the maximal vertex set in which each vertex has at least k incident s-cliques inside the set. These sets define the $(1, s)$-nucleus levels. Therefore $h^\star[v] = \kappa_s(v)$. The crude pass bound follows because across all vertices the sum of integer values can decrease at most $n\kappa_{\max}$ times before the final no-change pass. □ □

Theorem 5.1 (SPIN Complexity). *Algorithm 2 computes κ_s in $O(T \cdot \Sigma \cdot \log d_{\max})$ work, where $T \leq 1 + n\kappa_{\max}$ is the number of passes.*

Proof. Each pass calls VertexSupport(v, h) for every vertex v. The cost of one call is $O(\deg_\Sigma(v) \log \deg_\Sigma(v))$. Summing over vertices gives $O(\Sigma \log d_{\max})$ work per pass. Lemma 5.2 bounds the number of passes. □ □

5.3 Eliminating Redundant Sweeps

SPIN is useful because it exposes the equations, but its full passes recompute many vertices whose values can no longer change. Once a vertex reaches its final peel level, later events cannot increase it. SPIN* removes this redundancy by using the same nondecreasing peel order as classical core decomposition. The algorithm keeps the same value semantics as SPIN, but it updates them with local path events instead of global passes. For each path, it stores whether the path is still live, how many pivot vertices have not been finalized, and the fixed pivot requirement. It also keeps a finalized set and a bucket queue keyed by the current value. When one minimum vertex is popped, only paths containing that vertex can change. Theorem 4.1 says how each such path changes, and Lemma 4.1 gives the exact support loss for the unfinalized path-mates. Thus the invariant is local: unfinalized vertex values reflect the current live path-counter contributions, clipped below by the current peel level.

SPIN* is an optimization of SPIN, not a different decomposition objective. It keeps the same witness equation but changes the schedule. Instead of scanning every vertex after every pass, it updates only the vertices whose path contributions changed. Algorithm 3 outputs the core values and emits a sorted deletion log as a byproduct. Its running time depends on U, the number of incremental support-update events that actually occur, rather than on the number of full scans.

Algorithm 3 first initialises every path's pivot count $p_{\mathcal{P}}$, pivot requirement $\eta_{\mathcal{P}}$, and liveness bit (Lines 1-2), then sets each vertex's working value $h[v]$ to its initial s-clique support and inserts v into the corresponding bucket (Lines 3-4). The main loop (Lines 6-15) advances the peel level k to the smallest non-empty bucket and pops the next vertex v (Line 7), finalises $\kappa_s(v) \leftarrow k$ (Line 8), and walks every live path that contains v (Lines 10-13): if v is a hold, the path dies (Line 11); if v is a pivot, the path's pivot counter decrements and the path also dies once $p_{\mathcal{P}} < \eta_{\mathcal{P}}$ (Lines 12-13). Affected path-mates that have not been finalised are recomputed via Lemma 4.1 (Line 14) and moved to a lower bucket if their value dropped (Line 15). The pseudocode writes the affected update as VertexSupport to keep the fixed-point semantics visible; the implementation evaluates the changed contribution with the binomial loss formula rather than recomputing from scratch, so the total work scales with the number of incremental update events U rather than with the number of full scans. Buckets maintain the minimum-h vertex because values only decrease and are bounded by $\kappa_{\max}$ [1].

Lemma 5.3 (SPIN* Equivalence). *Algorithm 3 returns the same κ_s values as Algorithm 2.*

Algorithm 3: SPIN* — core values via single-pass bucket-queue peeling.

Input: Graph G, clique size s, CPI $\mathcal{P}$.
Output: $\kappa_s(v)$ for every $v \in V$.

foreach *path* $\mathcal{P} \in \mathcal{P}$ **do**
  $p_{\mathcal{P}} \leftarrow |V_p(\mathcal{P})|$; $\eta_{\mathcal{P}} \leftarrow s - |V_h(\mathcal{P})|$; alive$_{\mathcal{P}} \leftarrow (p_{\mathcal{P}} \geq \eta_{\mathcal{P}})$;
foreach $v \in V$ **do**
  $h[v] \leftarrow \deg^{(s)}(v)$; insert v into bucket $B[h[v]]$;
$k \leftarrow 0$; $V_{\text{fin}} \leftarrow \emptyset$;
while $|V_{\text{fin}}| < n$ **do**
  advance k until $B[k] \neq \emptyset$; pop v from $B[k]$;
  $\kappa_s(v) \leftarrow k$; append (v, k) to the deletion log; $V_{\text{fin}} \leftarrow V_{\text{fin}} \cup \{v\}$;
  Aff $\leftarrow \emptyset$;
  foreach $\mathcal{P}$ *with* alive$_{\mathcal{P}}$ *and* $v \in V_h(\mathcal{P}) \cup V_p(\mathcal{P})$ **do**
    Aff $\leftarrow$ Aff $\cup$ $((V_h(\mathcal{P}) \cup V_p(\mathcal{P})) \setminus \{v\})$;
    if $v \in V_h(\mathcal{P})$ **then**
      alive$_{\mathcal{P}} \leftarrow$ false; `// path death`
    else
      $p_{\mathcal{P}} \leftarrow p_{\mathcal{P}} - 1$;
      if $p_{\mathcal{P}} < \eta_{\mathcal{P}}$ **then** alive$_{\mathcal{P}} \leftarrow$ false;
  foreach $u \in$ Aff $\setminus V_{\text{fin}}$ **do**
    $y \leftarrow \max\{k, \text{VertexSupport}(u, h)\}$; `// implemented incrementally with Lemma 4.1`
    if $y < h[u]$ **then** move u from $B[h[u]]$ to $B[y]$; $h[u] \leftarrow y$;
return κ_s

Proof. The algorithms use the same witness equations. SPIN evaluates them by repeated full passes over the vertex set. SPIN* evaluates them in the standard nondecreasing peel order. The loop invariant for SPIN* is that every unfinalized vertex u has the same value it would obtain from the current live witness multiset under the fixed-point equations, except that values below the current peel level k are clipped to k. The invariant is true initially because $h[u] = \deg^{(s)}(u)$. When v is finalized, only paths containing v can change any remaining witness multiset. Theorem 4.1 gives the exact state transition for each such path. Lemma 4.1 gives the exact support loss for every affected path-mate. Thus the updates to Aff restore the invariant, while vertices outside Aff need no update. Popping a minimum bucket finalizes the next vertex whose fixed-point value cannot exceed the current level. By Lemma 5.2, that fixed point is κ_s. □ □

Theorem 5.2 (SPIN* Complexity). *Algorithm 3 runs in $O(\Sigma + U + \kappa_{\max} + n)$ time, where U is the total number of incremental support-refresh events over all affected path-mates.*

Proof. Initialization of vertex values and path counters costs $O(n + \Sigma)$. Each vertex–path incidence is visited when its vertex is finalized, so path-event processing costs $O(\Sigma)$. Each affected incremental refresh contributes one unit to U. Bucket moves and bucket pops are $O(1)$ amortized. The bucket cursor advances at most $\kappa_{\max}$ times. The total time is therefore $O(\Sigma + U + \kappa_{\max} + n)$. □ □

The mutable baseline pays heap and residual-index maintenance costs on the same logical peel events. The experiments in Section 8 measure the impact of replacing those operations with static path counters.

6 Parallel CPI Construction

In this section, we parallelise the CPI construction phase, which becomes the dominant remaining cost once SPIN* makes peeling

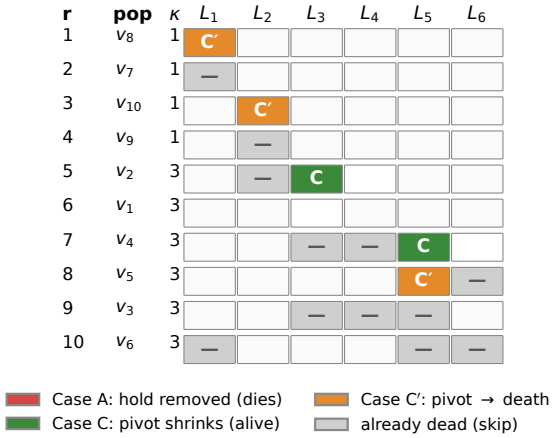

Figure 2: Incremental peel trace on the running example.

counter-only. We first explain why the seed loop admits parallel execution and how the static output contract makes the merge cheap, then present ParaBuild (Algorithm 4); the empirical scaling is reported in Section 8 (Table 6). The phase-breakdown tables in Section 8 measure SPIN*'s peel time separately from graph loading and CPI construction. They show that after SPIN*, peeling is often much smaller than construction. On `web-it-2004` at $s = 3$, SPIN* peels in 351 ms while loading and construction take about 127 seconds. On `web-Stanford` at $s = 60$, peeling takes 1.4 ms while loading and construction take about 6.1 seconds. This makes CPI construction the load-bearing cost of the static-state pipeline, so further pipeline-level speedups must address construction.

Why the seed loop can be parallel. The CPI builder follows the standard degeneracy-ordered Bron–Kerbosch search with Tomita pivoting [2, 4, 9, 13]. Each s-clique has one owner: the smallest vertex of the clique in the degeneracy order. Therefore, the outer loop over seed vertices partitions the emitted s-clique families. No two seeds own the same s-clique.

Static CPI also changes what construction must produce. The mutable baseline builds residual tree state because later peeling may edit that state. Our $r = 1$ pipeline never edits the stored hold or pivot sets. The builder therefore only has to emit a path list and vertex–path incidence arrays. This output can be accumulated in thread-local arenas and merged after all seeds finish.

Thread-local construction. ParaBuild takes G, s, and a thread count. Each worker owns a contiguous seed range, a local path arena, and a generation-stamped membership bitmap. This ownership is enough because seed ranges partition the emitted clique families. While a worker processes its seeds, it writes only to its own arena. After all workers finish, a deterministic merge concatenates the arenas and builds the shared incidence CSR. The expensive work is still the Bron–Kerbosch search under the seeds; the merge is linear in the emitted incidences.

Algorithm 4 computes the degeneracy ordering σ on G (Line 1) and partitions the seed range into T contiguous chunks (Line 2). Each thread allocates a private path arena and a generation-stamped membership bitmap (Lines 4-5), then walks its seed range (Lines 6-9): for every seed v it increments the bitmap generation (Line 7), restricts the neighbourhood to vertices later in σ (Line 8), and

Algorithm 4: ParaBuild — parallel CPI construction via per-thread path buffers.

Input: Graph G in CSR; clique size s; thread count T.
Output: Static CPI $\mathcal{P}$ in shared CSR.

```
1  compute degeneracy ordering σ on G;
2  launch T threads and assign each thread t a contiguous chunk σ_t of seed
     vertices;
3  foreach thread t ∈ {1, ..., T} in parallel do
4      A_t ← ∅ ;                                  // thread-local path arena
5      b_t ← generation-stamped bitmap over V;
6      foreach seed vertex v ∈ σ_t do
7          increment generation of b_t;
8          N⁺ ← {u ∈ N_G(v) | σ(u) > σ(v)};
9          BkEmit(v, N⁺, A_t, b_t) ;   // pivoted BK on G[N⁺], with v
             as a hold
10 P ← deterministic concat-and-prefix-sum merge of {A_t}_{t=1}^T;
11 build incidence CSR over P in parallel;
12 return P
```

runs the pivot-aware enumeration BkEmit streaming each emitted path into the thread-local arena (Line 9). After all threads finish, a deterministic concat-and-prefix-sum merge concatenates the arenas (Line 10) and builds the shared incidence CSR in parallel (Line 11); the final static CPI is returned (Line 12). BkEmit is the same pivot-aware enumeration used by the CPI construction lineage [6, 9]; the difference is that it streams each emitted path into A_t instead of storing a mutable recursion tree for later residual updates.

Correctness. ParaBuild is a parallel schedule of the same seed-owned enumeration. Seed ownership gives disjoint emitted s-clique families. The deterministic merge changes only the storage order, not the represented path families. The resulting path set satisfies Definition 2.5. Therefore, the support formula of Lemma 2.1 applies unchanged.

Scope. The design parallelizes construction because construction is the phase exposed by the static-CPI pipeline. It does not claim a lower bound showing that SPIN* cannot be parallelized. It also does not claim ideal scaling. The scaling evidence is reported in Table 6.

7 Core-Value Hierarchy Construction

In this section, we extract the $(1, s)$-clique-connected nucleus hierarchy of Definition 2.3 from the preserved CPI of Section 4, after SPIN* has computed the core values. We first show that each leaf can be treated as a hyperedge that is born at the minimum core value among its stored vertices, then present BuildHier (Algorithm 5) as a single descending union–find scan over leaves; the cost is $O(\Sigma \log \Sigma)$ and no clique is re-enumerated.

The core numbers define a nested family of vertex sets. For any level k, the super-level set is $V_k = \{v \mid \kappa_s(v) \geq k\}$. As k decreases, vertices enter the set. Definition 2.3 measures connectivity inside V_k through s-clique witnesses contained in V_k. BuildHier constructs the elder-rule join tree of this clique-witnessed filtration directly from CPI leaves.

Leaves as connectors. A CPI leaf P encodes the family of s-cliques $V_h(P) \cup X$ for every η_P-subset $X \subseteq V_p(P)$ (Definition 2.5). Define the *birth level* of P as

$$k_P = \min_{v \in V_h(P) \cup V_p(P)} \kappa_s(v).$$

At level $k \leq k_P$ every leaf vertex has core value at least k, so every encoded s-clique is contained in V_k. Any two vertices of the leaf either lie inside one such clique (when $\eta_P \geq 2$) or are linked by a length-two path through a vertex of $V_h(P)$ (when $\eta_P = 1$ and $V_h(P)$ is non-empty); for $V_h(P) = \emptyset$ and $\eta_P \geq 2$, any pair appears together in some s-subset. In every case the leaf vertices form a single s-connected component the moment P becomes active. This lets BuildHier treat each leaf as a single hyperedge in a union–find structure: the leaf node is unified with every vertex it contains. Two vertices then end up in the same component if and only if they are s-connected through active CPI cliques at the current level.

Algorithm 5: BuildHier — elder-rule join tree atop SPIN*.

Input: Graph G, clique size s, CPI $\mathcal{P}$.
Output: Elder-rule join tree T of the s-clique-connected super-level filtration.

```
1  κ_s ← SPIN*(G, s, P) ;                                  // Algorithm 3
2  foreach leaf P ∈ P do
3      k_P ← min{κ_s(v) | v ∈ V_h(P) ∪ V_p(P)};
4  sort P by nonincreasing k_P (ties broken by leaf id);
5  initialise union-find U over V ∪ {P | P ∈ P}; birth[r] ← +∞ for every
     root r;
6  T ← empty forest;
7  foreach leaf P in sorted order do
8      R ← {Find(x) | x ∈ V_h(P) ∪ V_p(P) ∪ {P}};
9      foreach root r ∈ R do
10         if birth[r] = +∞ then
11             emit a hierarchy node n_r with k_birth ← k_P; birth[r] ← k_P;
12     if |R| ≥ 2 then
13         sort R descending by (birth[r], |r|); let r* be the elder ; // elder
             rule
14         foreach r ∈ R \ {r*} do
15             attach n_r as child of n_{r*} at level k_P in T ; // younger dies
16         Union all of R into a single component;
17 return T
```

Reverse scan over leaves. BuildHier processes leaves in nonincreasing k_P, as the level descends through the filtration. A union–find structure spans both vertices and leaf identifiers; processing leaf P at its birth level fires unions Union(P, v) for every $v \in V_h(P) \cup V_p(P)$. A birth level is recorded for each component root, so a merge applies the elder rule. Vertices not appearing in any leaf with $k_P \geq 1$ have $\kappa_s(v) = 0$ and are omitted from the hierarchy.

Algorithm 5 first invokes SPIN* to compute the core values (Line 1), then assigns each leaf its birth level k_P as the minimum core value among its hold and pivot vertices (Lines 2-3). Leaves are sorted by nonincreasing k_P (Line 4) so they fire in descending peel order, and the union–find structure is initialised over both vertices and leaf identifiers with all roots born at $+\infty$ (Line 5). For each leaf P in sorted order (Lines 7-13), the algorithm collects the current root set R of every vertex in P together with the leaf itself (Line 8), emits a fresh hierarchy node for any root that has not been born yet (Lines 9-11), applies the elder rule when $|R| \geq 2$ by selecting the earliest-born root $r^\star$ and attaching the others as its children at level k_P (Lines 12-14), and finally unifies all roots into a single component (Line 15). The returned forest T is the elder-rule join tree.

Correctness. The descending scan visits each leaf at exactly the level it first contributes s-cliques contained in the current super-level set. At that level, every vertex it contains lies in the super-level set, so the leaf witnesses pairwise s-connectivity for all of them through a common element of $V_h(P)$ (or directly within an s-subset when $V_h(P) = \emptyset$). Conversely, every active s-clique at level k belongs to some leaf with $k_P \geq k$, and that leaf has already fired

Table 2: Hierarchy-extraction time on `com-dblp`: BuildHier vs. snapshot baseline.

s	BuildHier (ms)	snapshot (ms)	speedup
3	358.7	588.9	1.64×
5	129.6	432.0	3.33×
10	13.4	32.7	2.44×
15	4.1	10.0	2.44×

before the scan reaches level k; so every s-clique-connected merge of Definition 2.3 is performed. When two distinct components meet at a leaf, the elder rule preserves the component with earlier birth and attaches the younger one in the join tree [3].

Complexity. Computing every k_P and sorting leaves costs $O(\Sigma + L \log L)$, where $L = |\mathcal{P}|$. Each leaf fires at most $|V_h(P)| + |V_p(P)|$ union-find operations, summing to $O(\Sigma\, \alpha(n + L))$ over all leaves. Total time $O(\Sigma \log \Sigma)$, dominated by the sort, with no clique re-enumeration: BuildHier reads each CPI leaf at most twice (once to compute k_P, once to fire unions).

Memory. Beyond the union-find of size $n + L$, the routine needs a list of vertices per leaf to fire unions. SPIN*'s vertex-to-leaf inverse CSR (Section 5) supports a single bucket-CSR pass that materialises the per-leaf lists in $O(\Sigma)$ temporary memory; this scratch is freed on return. The static CPI budget of Section 8 therefore holds throughout peeling proper and is only briefly elevated during the hierarchy emit.

Consequence. The clique-coherent core-value hierarchy is not a separate decomposition problem in this pipeline. It is a byproduct of the static CPI surviving peeling: leaves act as hyperedges of an implicit s-clique witness graph, and a single descending pass with union–find produces the join tree. Methods that mutate the CPI during peeling cannot reuse it for hierarchy emit and need a second clique-enumeration pass [11].

Empirical cost. Table 2 compares BuildHier against a level-by-level snapshot baseline on `com-dblp` (n=317k vertices, m=1.05M edges), with three runs per cell. The snapshot baseline runs one SPIN* decomposition per requested level and writes the join tree from the snapshots; it is the natural alternative when the static CPI is not preserved. BuildHier is faster on every reported s and the gap widens with s: at s=5 the speedup is 3.33× (median 129.6 ms versus 432.0 ms), and the absolute cost stays below 360 ms even at s=3 where Σ is largest. Scaling at s=5 is sublinear in $|E|$: keeping 25%/50%/75%/100% of the edges costs 67/103/123/127 ms, consistent with the $O(\Sigma \log \Sigma)$ bound being dominated by the leaf sort.

8 Experiments

The previous sections prove the static-CPI invariant and derive two ways to compute the same fixed point: the direct SPIN solver and the practical SPIN* realization. The comparison with CND is therefore a cost comparison, while exactness comes from the invariants and fixed-point equivalence proved earlier. They ask whether the static pipeline changes the cost profile enough to matter. **RQ1** measures end-to-end time and peak memory for SPIN*, the practical static-CPI pipeline. **RQ2** separates construction from peeling, because counter-only peeling shifts rather than removes all cost. **RQ3** asks whether the static construction contract gives useful parallelism. **RQ4** measures SPIN as the direct fixed-point baseline. **RQ5** probes dense synthetic graphs where clique counts grow quickly. **RQ6** pushes the pipeline to a billion-edge graph (`com-friendster`, 1.8B edges). **RQ7** isolates the contribution of each pipeline idea. **RQ8** sweeps a vertex-induced subsampling ratio to test how runtime and memory scale with input size.

Table 3: Datasets.

Graph	$\lvert V\rvert$	$\lvert E\rvert$	avg. deg.
`com-amazon` [14]	334,863	925,872	5.5
`com-dblp` [14]	317,080	1,049,866	6.6
`twitter` [14]	81,306	1,342,296	33.0
`web-Stanford` [14]	281,903	1,992,636	14.1
`com-youtube` [14]	1,134,890	2,987,624	5.3
`web-Google` [14]	875,713	4,322,051	9.9
`wiki-Talk` [14]	2,394,385	4,659,565	3.9
`web-it-2004` [14]	509,338	7,178,413	28.2
`soc-pokec` [14]	1,632,803	22,301,964	27.3
`com-orkut` [14]	3,072,441	117,185,083	76.3
`com-friendster` [14]	65,608,366	1,806,067,135	55.1

Hardware and software. All experiments run on a dual-socket Intel Xeon Gold 6248*R* machine with 24 cores and 503 GB DDR4 memory. The operating system is Ubuntu 22.04. The code[1] is compiled with Clang 18.1 using `-O3 -march=native`. It is linked against TBB 2021.10, Boost 1.83, and `robin_hood::unordered_flat_map`. Single-threaded measurements use one core. Parallel-construction measurements use the thread counts reported in Table 6. Time and memory measurements are averaged or summarized over repeated runs as stated in each experiment.

Datasets. Table 3 lists the benchmark graphs. The set covers collaboration, e-commerce, social, and web graphs. The largest graph is `com-friendster` with 1.8B edges. The benchmark sweep on the ten smaller graphs yields 307 matched cells where both SPIN* and CND complete successfully, and the headline speedup numbers are reported over the matched subset; `com-friendster` is run standalone (Section 8) because CND cannot complete on it.

Algorithms and methodology. CND is the mutable-CPI implementation of general (r, s)-nucleus decomposition [9], restricted to $r = 1$ for this comparison. It maintains the CPI as mutable residual state: each peel step deletes the popped vertex from every path that references it, rewrites the affected path records, and updates a hash-indexed reverse map so the next pop can locate its incident paths. The dominant per-pop cost is therefore the residual-index rewrite plus the hash-set rebuild that reflects the deletion; this work is intrinsic to the mutable design and is what SPIN* avoids by leaving the CPI unchanged. SPIN* is the practical static-CPI pipeline. It constructs the static index once and computes core values by incremental path-counter updates. SPIN is evaluated separately because it is the direct fixed-point formulation rather than the practical implementation. All three solvers consume the same input edge list and run on the same hardware. The comparison isolates the cost of CPI mutability against static-state peeling, not differences in input processing or execution environment.

For every matched $(graph, s)$ pair, we run CND and SPIN* on the same input. We record the end-to-end time and the program's memory usage. End-to-end time includes graph loading, degeneracy ordering, CPI construction, peeling, and output emission.

[1] We will release the code, bench harness, and result CSVs under an anonymised public archive at submission and de-anonymise the repository on acceptance.

Table 4: Per-incidence memory budget at $\Sigma \gg n$.

Component	Baseline	SPIN*
BK recursion tree T	8Σ	0
Path-to-vertex storage (V_h, V_p)	0	4Σ
Per-path vertex set H_P	32Σ	—
Reverse vertex-to-path map	48Σ	—
Vertex-to-path incidence	—	4Σ
Support array	$4n$	$4n$
Heap / bucket array	$24n$	$16n + 8\kappa_{\max}$
Analytical Σ-coefficient	$\approx 88\Sigma$	$\approx 10\Sigma$
Measured: com-youtube, s=5	642 MB	315 MB
Measured: web-it-2004, s=3	556 MB	425 MB
Measured: web-Stanford, s=10	261 MB	176 MB

The exactness claim comes from Theorem 4.1, Lemma 5.2, and Lemma 5.3.

RQ1: end-to-end time and memory. The benchmark has 307 matched cells where both SPIN* and CND complete. Across the ten graphs, SPIN* achieves a peel-time speedup over CND of geometric-mean 13.50× and maximum 44.76×. The end-to-end speedup is smaller (gmean 1.10×) because the shared SDCT construction phase dominates total runtime once peeling is cheap; this is the bottleneck shift that motivates ParaBuild in Section 6. The memory-usage ratio (CND over SPIN*) has geometric mean 1.55× and maximum 3.92×. Figure 3 and Figure 4 show where these gains occur.

The memory improvement is explained by the incidence-level budget in Table 4. The mutable baseline stores path hash sets and reverse hash maps. SPIN* stores flat path-to-vertex and vertex-to-path arrays plus path counters. The analytical per-incidence gap is larger than the observed memory gap because graph adjacency, allocator overhead, and per-vertex structures do not scale with Σ.

RQ2: where does the time go? The static invariant makes peeling cheap, so the next question is whether construction becomes the dominant cost. The phase study uses com-youtube, web-Stanford, and web-it-2004. It reports median times over three runs. Table 5 reports representative per-phase time medians (*load*/*build*/*peel* in ms) alongside memory usage (in MB) for both algorithms, over three runs.

The phase data supports the bottleneck-shift story. Peeling falls to tens or hundreds of milliseconds on many cells. Construction then becomes a major remaining cost, especially on the web graphs. This is why ParaBuild is part of the algorithmic pipeline rather than a detached engineering detail.

RQ3: does static construction parallelize? ParaBuild exposes seed-level parallelism because each seed can stream its emitted paths into thread-local storage. Table 6 reports build time as the thread count grows from 1 to 24. Each cell is the median of three runs. The table shows strong scaling on large web and social graphs and weaker scaling on smaller overhead-dominated cases.

The correct interpretation is not that construction is perfectly parallel. It is that static output removes hot-path residual-index mutation from the seed walks. This lets the expensive enumeration work scale until memory bandwidth and scheduling overhead become visible.

RQ4: how much does SPIN* optimize over SPIN? SPIN is the literal single-thread solver for the same equations that SPIN* realizes by deletion events. The two algorithms compute the same core values (Lemma 5.3), so the comparison isolates the effect of replacing repeated full-graph passes with incremental updates. Both solvers read paths from the same static index; the SPIN reference therefore reflects the algorithmic cost of repeated full-pass fixed-point iteration, not a difference in how the index is stored. The two curves are already shown in Figure 3: SPIN* is the solid blue line and SPIN is the dotted-triangle line on every panel. Across the 291 matched $(graph, s)$ cells, SPIN* delivers a 36.2× geometric-mean peel-time speedup over SPIN, with maximum 2,369× on web-it-2004 at $s = 53$, where SPIN repeatedly sweeps every vertex while SPIN* finishes after a few path-event updates. On the easiest cells SPIN* can be slightly slower (minimum ratio 0.62×) because the bucket-queue setup and live-path bookkeeping add fixed overhead that SPIN's tight inner loop avoids when one or two passes already converge. On the largest inputs SPIN hits the 1h cap (com-orkut for $s \geq 3$, wiki-Talk for the single cell $s = 10$); SPIN* finishes the same configurations in seconds to minutes.

SPIN's memory profile is essentially identical to SPIN*'s on every paired cell, because both share the same static index: the geometric-mean memory ratio (SPIN/SPIN*) is 0.97 over the 291 matched cells, and SPIN is slightly smaller than SPIN* on 83.8% of them. The reason is structural: SPIN*'s extra state beyond the shared CPI consists of a per-path counter cache ($\sim$ 13 bytes per leaf) and a peel-order bucket queue ($\sim$ 17 bytes per vertex), which SPIN does not maintain because it scans every vertex each round. The sub-percent memory advantage of SPIN is therefore the price SPIN* pays to avoid repeated full-graph passes; the trade is the textbook "space for time" choice, with the time term dominating by orders of magnitude.

The result is not a failure of the fixed-point view. It explains why the paper needs both algorithms. SPIN gives the direct semantic formulation. SPIN* computes the same fixed point by processing only the vertices whose witness counters changed.

RQ5: dense synthetic stress. The benchmark graphs are sparse. The stress test follows the dense synthetic protocol used in prior nucleus-decomposition studies [9, 10]. It generates 1000-vertex graphs by incremental clique injection and sweeps 12 densities from 0.01 to 0.60 for $s \in \{5, 7, 9, 11\}$. Both methods grow quickly as density and s increase. Where both implementations finish, SPIN* remains below CND in time and memory.

RQ6: scaling to a billion-edge graph. The ten-graph benchmark stops at com-orkut (117M edges); to push the pipeline an order of magnitude further, we run SPIN* with T=24 on com-friendster (65M vertices, 1.806B edges, the largest SNAP graph available). CND cannot complete on this graph at any s because its mutable residual state grows past the available 503 GB at construction. Table 7 reports the per-phase time, the encoded clique count Σ, and memory usage for the configurations on which SPIN* produces a complete decomposition. SPIN* successfully decomposes the graph for a wide range of s: at s=6 it indexes $\Sigma \approx 2.26 \times 10^{10}$ encoded s-clique witnesses and peels that entire witness mass in 651 ms; at s=11 it indexes $\Sigma \approx 1.68 \times 10^{10}$ witnesses and finishes peeling in 167 ms. Construction is what dominates total runtime on this graph (about 5–7 thousand seconds against under one second of

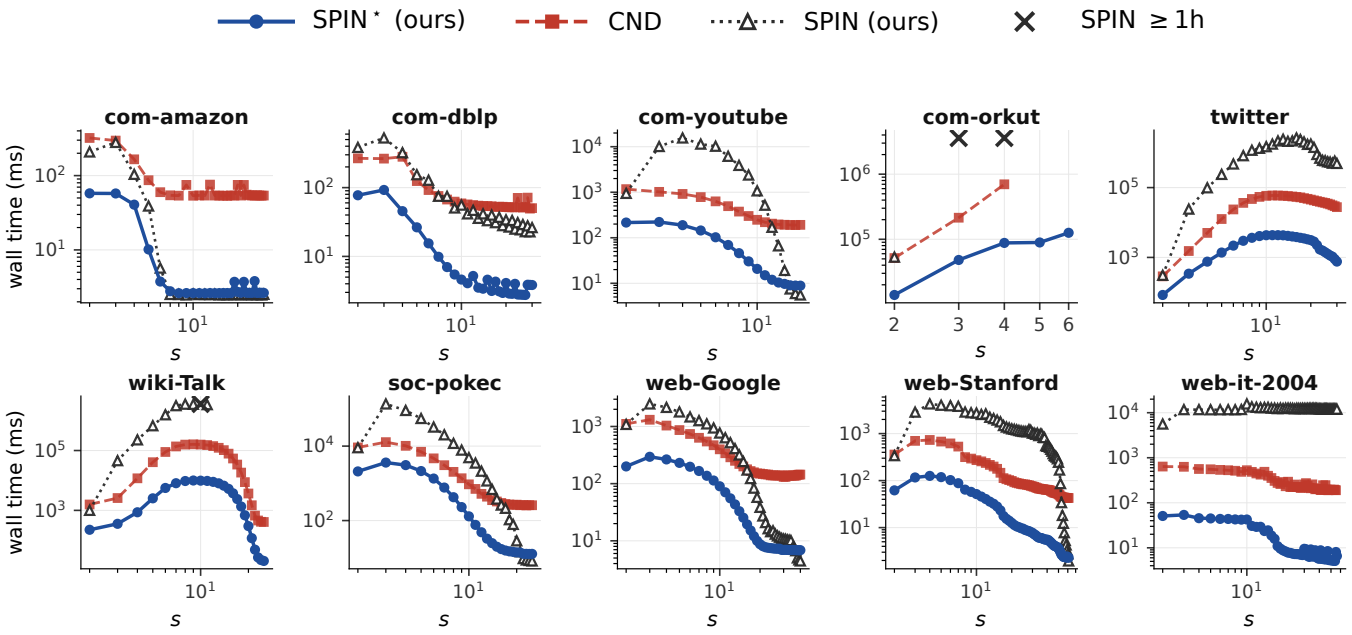

Figure 3: End-to-end time across ten graphs.

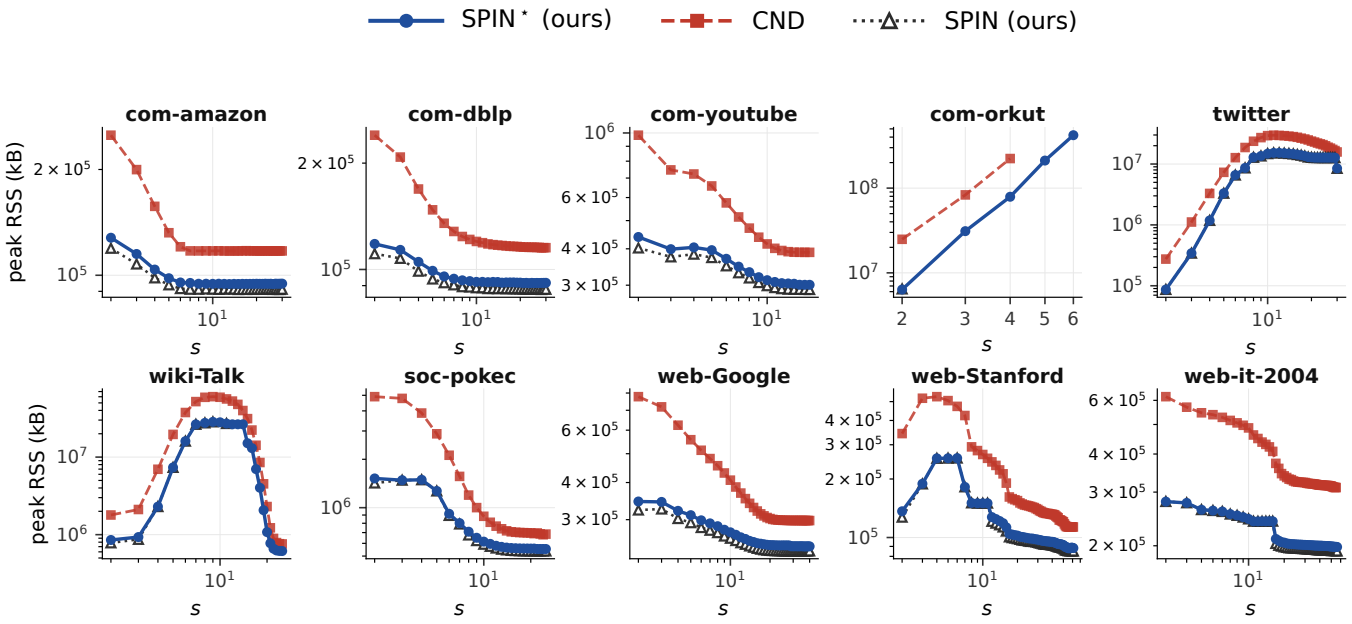

Figure 4: Memory usage across ten graphs.

peel), confirming that the bottleneck shift carries over to the billion-edge regime; the parallel builder of Section 6 is what makes the construction phase tractable on a 1.8B-edge input.

RQ7: where do the gains come from? The SPIN* pipeline combines two algorithmic ideas: the static-CPI invariant (Section 4), which removes residual maintenance from the peel loop, and a bucket-queue peel with per-event support updates (Section 5.3), which removes redundant full-graph passes. We isolate the two algorithmic ideas against the mutable baseline on the same $(graph, s)$ cells. **ST** runs the same fixed-point solver as CND but reads paths from an immutable static CPI; the solver still scans every vertex per round. **ST+bucket queue+events** is the full SPIN* pipeline (Algorithm 3). Table 8 reports median peel time at each step on the 171-cell matched subset, together with the gmean speedup over the previous step. Static state alone already gives a 3.80×

Table 5: Per-phase time (ms) and memory usage (MB).

Graph	s	SPIN* time (ms)			CND time (ms)			memory (MB)		mem
		load	build	peel	load	build	peel	SPIN*	CND	ratio
com-youtube	3	2081	3863	440.1	1812	3982	789.7	381	729	1.91×
com-youtube	5	2028	3730	273.5	1732	3795	581.6	315	642	2.04×
com-youtube	8	1967	2991	86.3	1638	3053	270.2	187	460	2.45×
com-youtube	12	1689	2701	13.0	1640	2649	148.5	119	385	3.25×
com-youtube	16	1640	2458	8.2	1964	2488	131.8	114	380	3.33×
web-Stanford	3	1251	6602	381.6	1250	6548	521.5	338	508	1.50×
web-Stanford	10	975	5840	130.4	1225	5961	210.1	176	261	1.48×
web-Stanford	20	999	5482	28.9	1228	5591	77.8	97	151	1.55×
web-Stanford	40	1224	5087	9.6	1034	5007	49.6	70	129	1.85×
web-Stanford	60	1224	4900	1.4	1224	4907	31.3	53	111	2.09×
web-it-2004	3	3389	124074	351.0	3378	125347	465.9	425	556	1.31×
web-it-2004	30	3372	122558	63.9	3037	122689	173.7	230	314	1.37×
web-it-2004	100	3407	121891	40.1	3377	121372	135.9	207	296	1.43×
web-it-2004	200	3416	113825	26.1	3406	107516	108.5	178	273	1.53×
web-it-2004	400	3409	7850	4.6	3368	7847	62.7	139	226	1.63×

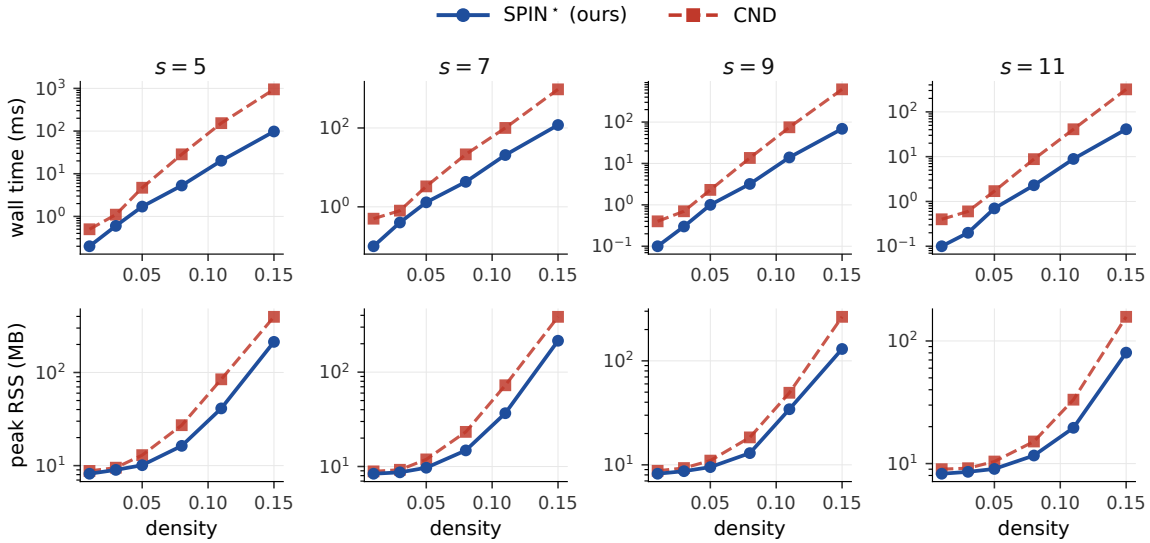

Figure 5: Dense-synthetic stress: time (top) and memory usage (bottom).

Table 6: ParaBuild **build time vs thread count.**

Graph	s	$T=1$	$T=2$	$T=4$	$T=8$	$T=16$	$T=24$	sp.
com-dblp	3	205	131	84	61	56	58	3.5×
com-dblp	15	182	115	73	51	43	45	4.0×
dblp-core30	3	47	36	28	25	22	22	2.1×
com-youtube	3	2644	1516	833	465	275	214	12.4×
com-youtube	16	2418	1367	744	399	224	168	14.4×
web-Stanford	3	5357	2776	1538	834	458	331	16.2×
web-Stanford	60	5196	2697	1483	793	422	297	17.5×
web-it-2004	3	92312	49798	26304	13048	5789	3946	23.4×
web-it-2004	400	83795	45977	24673	10935	5863	3930	21.3×

Table 7: Build/peel time, encoded clique count, and memory usage on `com-friendster` (1.806B edges).

s	build (s)	peel (s)	Σ	memory (GB)
2	5012	0.34	3.48×10^9	153
3	5579	0.56	8.94×10^9	324
4	5841	0.68	1.38×10^{10}	451
5	6220	0.62	1.90×10^{10}	446
6	6855	0.65	2.26×10^{10}	430
11	6470	0.17	1.68×10^{10}	297

gmean speedup over the mutable baseline: leaving the paths unchanged removes the residual-rewrite work that dominates CND.

Table 8: Pipeline ablation: median peel time on 171 matched cells.

Pipeline step	median peel (ms)	gmean speedup
CND (mutable baseline)	464.9	—
+ static CPI (full-pass solver)	105.3	3.80×
+ bucket queue + events (SPIN*)	14.8	3.95×
Cumulative CND → SPIN*		15.00×

Adding bucket-queue order with event-driven updates gives a further 3.95× over static-only; ordered finalisation eliminates redundant per-round vertex scans. The cumulative gmean speedup of the full pipeline over CND on these matched cells is 15.00×, consistent with the 13.50× headline reported in RQ1 over a larger and partly disjoint cell set. The two ideas compound rather than substitute: bucket-queue order without the static invariant cannot avoid re-deriving residual support after each peel, and the static invariant without ordered finalisation regresses to the full-pass behaviour measured for SPIN in RQ4.

RQ8: scaling with input size. The benchmark cells fix a graph and sweep s; they do not test how runtime and memory respond to growing the input itself. To probe scaling, we shrink each graph by vertex-induced subsampling: for each ratio $\rho \in \{0.2, 0.4, 0.6, 0.8, 1.0\}$ we keep $\lceil \rho\, n \rceil$ vertices selected uniformly at random with a fixed seed and take the induced subgraph (only edges with both endpoints kept). The same seed makes the subsamples nested ($20\% \subset 40\% \subset \cdots \subset 100\%$), so runtime and memory are monotone in ρ by construction. We pick vertex-induced rather than random-edge subsampling because SPIN*'s work scales with the encoded s-clique count Σ; removing edges uniformly drops Σ super-linearly because each missing edge can destroy many cliques, producing jagged curves, while a vertex-induced subsample preserves clique structure inside the kept vertex set. We re-run SPIN* on five graphs spanning two orders of magnitude in edge count (`com-dblp`, `com-youtube`, `web-Stanford`, `web-Google`, `soc-pokec`) for $s \in \{2, 3, 5, 7, 9, 11, 13, 15\}$ on each subsample. Figure 6 reports end-to-end time (top row) and memory usage (bottom row) as the ratio grows. On every graph SPIN*'s curves are ordered cleanly by ratio at every s in both metrics: denser subsamples take longer and use more memory, with no inversions, indicating that the static-CPI pipeline scales smoothly with input size from sparse collaboration graphs to dense social graphs.

Limitations. The static-CPI invariant is specific to r=1. For general (r, s) with $r \geq 2$, edge or higher-order peel events can destroy the cliques that a path stores, and the present pipeline does not subsume the mutable-CPI machinery of [9]. Memory is the binding resource on the densest graphs: Σ grows superpolynomially with s on dense inputs, so on a fixed-memory machine the largest reachable s is determined by the available RAM rather than by an algorithmic limit. Support counters use $\binom{p}{\eta}$ which exceeds 2^{63} at moderate p; the implementation guards the threshold pass with 128-bit accumulators when the count overflows 64 bits, but applications that need exact support magnitudes (rather than ranks) must use the same guard. Finally, the experiments here run single-machine; distributed-memory deployments and dynamic-graph updates are not evaluated.

Reproducibility. All experiments are scripted: build commands, dataset URLs, and bench drivers are version-controlled alongside the source. Each binary run is wrapped with `/usr/bin/time -v` so memory usage, page faults, and exit status are captured in every CSV row.

Takeaway. The experiments support the paper's dependency chain. SPIN* is faster and uses less peak memory than CND on the 307 matched benchmark cells. The phase breakdown shows that construction becomes a major remaining cost after peeling becomes counter-only. ParaBuild addresses that shifted bottleneck by parallelizing static construction. SPIN confirms why the direct fixed-point view is useful for semantics but unsuitable as the practical solver.

9 Case Studies

The experiments measure whether the static pipeline changes cost and scalability. They do not by themselves explain whether the specialized $r = 1$ case is worth optimizing. This section therefore asks a practical question: when an analyst wants vertex-level cohesive structure, is $(1, s)$ enough? On the ground-truth tasks reported here, $(1, s)$ matches the quality of higher-r nuclei on the evaluated metrics while costing much less. The claim is empirical and scoped to these datasets and metrics.

Table 9: Active set on `com-dblp` as s grows.

s	nonzero $\lvert V \rvert$	% of n	max core	top-100 min	top-1k min
2	317,080	100.00	1.13×10^{2}	1.13×10^{2}	3.10×10^{1}
3	269,181	84.89	6.33×10^{3}	6.33×10^{3}	4.65×10^{2}
4	194,957	61.49	2.34×10^{5}	2.34×10^{5}	4.50×10^{3}
5	127,805	40.31	6.44×10^{6}	6.44×10^{6}	3.15×10^{4}
8	36,412	11.48	3.86×10^{10}	3.86×10^{10}	2.63×10^{6}
10	19,354	6.10	5.97×10^{12}	5.97×10^{12}	2.02×10^{7}
15	6,767	2.13	2.74×10^{17}	2.74×10^{17}	2.65×10^{8}
20	3,396	1.07	1.69×10^{21}	1.69×10^{21}	1.41×10^{8}
25	2,069	0.65	2.18×10^{24}	2.18×10^{24}	2.63×10^{6}
30	1,358	0.43	7.61×10^{26}	7.61×10^{26}	4.65×10^{2}

Table 10: Ground-truth quality on `com-dblp`.

Method	top-K	Precision	rec50	tri/v
k-core	10,000	0.504	94	112.3
$(1, 3)$-core	10,000	0.523	114	113.4
$(1, 5)$-core	10,000	0.523	114	113.4
$(1, 10)$-core	10,000	0.523	114	113.4
$(1, 15)$-core	6,767	**0.548**	65	**154**
$(1, 30)$-core	1,358	0.507	4	**523**

Setup. For cross-r comparisons, we use the standard (r, s)-nucleus parameterization [11]. The paper's algorithmic results apply only to $r = 1$. The $r = 2$ and $r = 3$ results are baselines from the same nucleus family. Because those baselines score edges or triangles, we project their scores to vertices by assigning each vertex the maximum score of an incident r-clique. Vertices are ranked by projected score in descending order, with ties broken uniformly.

The ground-truth datasets are `com-dblp` and `com-youtube` from SNAP [14]. `com-dblp` has 317,080 vertices, 1,049,866 edges, and 5,000 venue-based ground-truth communities. `com-youtube` has 1,134,890 vertices, 2,987,624 edges, and 5,000 ground-truth communities.

We use three ranking metrics. Precision@K is the fraction of top-K vertices that belong to at least one ground-truth community. rec50 is the number of communities with at least 50% of their vertices inside the top-K. Triangle density is the number of triangles per vertex in the induced top-K subgraph.

Case study 1: s is a granularity knob. The first question is what s changes when $r = 1$. On `com-dblp`, increasing s sharply shrinks the active set while preserving the very top of the ranking. Table 9 reports the active-set size and top-prefix thresholds.

The active set shrinks from all vertices at $s = 2$ to 1,358 vertices at $s = 30$. This is a 234× reduction. At the same time, the maximum core value spans many orders of magnitude because high-s supports are binomial counts. The practical interpretation is simple. s controls how aggressively the ranking filters the graph.

Case study 2: $(1, s)$ improves the k-core ranking. On `com-dblp`, the first non-trivial clique support already improves the ground-truth ranking over k-core. Table 10 reports top-10,000 quality.

$(1, 3)$-core raises precision from 0.504 to 0.523 and rec50 from 94 to 114. For $s \in \{3, 5, 10\}$, the top-10,000 quality is identical in this table. For larger s, the active set drops below 10,000, so the comparison changes from ranking quality at fixed K to density

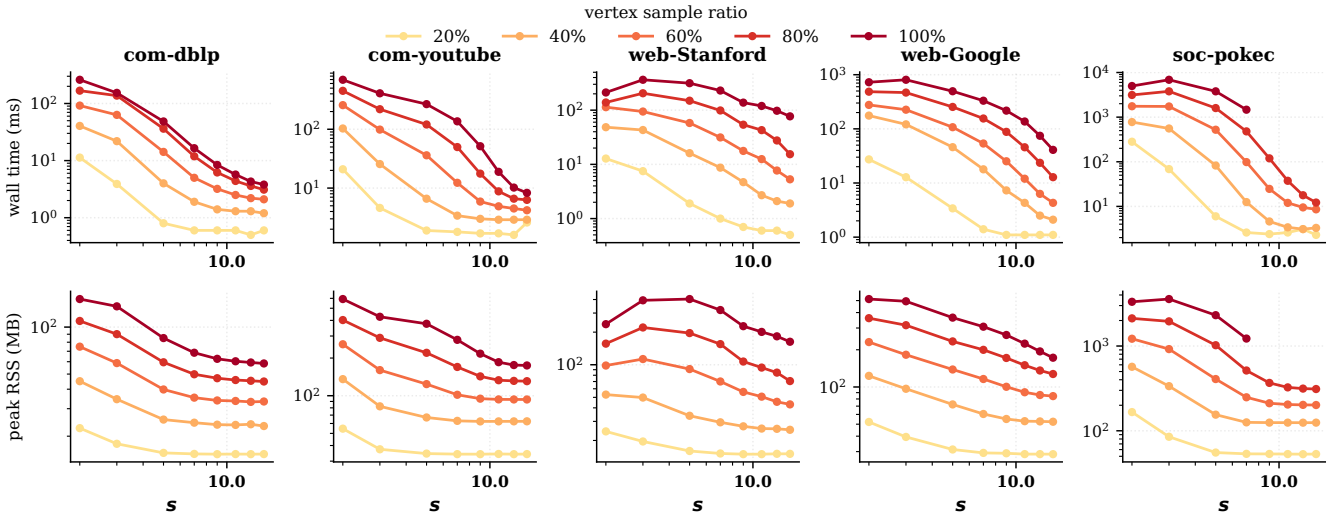

Figure 6: Vertex-induced scalability: time (top) and memory usage (bottom).

Table 11: Cross-r quality and runtime on `com-dblp` ($s = 10$).

Method	Precision	rec50	tri/v	Time (ms)
k-core	0.504	94	112.3	2.3
$(1, 10)$-core	0.523	114	113.4	13.4
$(2, 10)$-core	0.529	113	71.8	123.1
$(3, 10)$-core	0.524	114	113.4	2,379.6
(1) vs. (3)	−0.001	0	0.0	−178×

of the surviving subgraph. $(1, 30)$-core reaches 523 triangles per vertex on 1,358 vertices.

Case study 3: higher r does not buy quality on the studied graphs. Whether edge- or triangle-level nuclei justify their cost is the next question. We fix s and compare $(1, s)$, $(2, s)$, $(3, s)$ on two graphs from different domains: `com-dblp` (Table 11) and `com-youtube` (Table 12). Across both graphs the $(1, s)$ ranking matches higher r on rec50 and triangle density while running 67–178× faster. On `com-dblp` at s=10, $(1, 10)$ and $(3, 10)$ agree on rec50 (114 vs. 114) and triangle density (113.4 vs. 113.4) while the former is 178× faster. On `com-youtube` the same pattern holds for every $s \in \{5, 10, 15\}$, with r=1 at most a single rec50 point below r=3 and between 6.2 and 67.5× faster.

Across the broader DBLP grid the $r = 1$ points dominate the $r \in \{2, 3\}$ points on the quality–runtime tradeoff: they match higher-r rec50 and triangle density while running an order of magnitude faster. This does not prove higher r is never useful; it shows that on this vertex-ranking task, increasing r mostly adds cost.

The DBLP picture is not a collaboration-data artifact. Table 12 reports the same cross-r comparison on `com-youtube` across three values of s; the speedup factors and quality matches are listed above.

Across all three s values, $r = 1$ reaches the same rec50 and triangle-density contours as higher-r settings (see Table 12) with lower runtime; the same pattern holds on `com-youtube` as on `com-dblp`.

Case study 4: s changes local resolution. The previous studies use global rankings. The ego-network visualization asks whether s

Table 12: Cross-r quality and runtime on `com-youtube`.

Method	rec50	tri/v	Time (ms)	vs. $(1, s)$
$(1, 5)$	283	173	274	—
$(2, 5)$	286	161	2,580	9.4×
$(3, 5)$	288	159	13,112	47.9×
$(1, 10)$	49	293	31	—
$(2, 10)$	49	293	319	10.3×
$(3, 10)$	49	293	2,093	67.5×
$(1, 15)$	0†	173	9	—
$(2, 15)$	0†	173	55	6.2×
$(3, 15)$	0†	173	87	9.8×

†At $s = 15$, the active set has 95 vertices, below the smallest SNAP community size, so rec50 is zero for every method.

Table 13: Ego-network statistics around a `com-dblp` anchor.

s	$\lvert V \rvert$	CCs	largest CC	intra-frac	avg cond
3	462	1	462	0.992	0.004
5	335	5	329	0.796	0.121
10	66	26	32	0.340	0.636

also changes local interpretability. Figure 7 shows the two-hop ego-network of `com-dblp` vertex 52213 under $s \in \{3, 5, 10\}$. Table 13 reports the component statistics.

At $s = 3$, the ego-network remains one large component. At $s = 10$, it becomes 26 smaller components. This behavior makes s a local resolution knob.

Case study 5: hierarchy profiles differ by domain. BuildHier turns the core values into a $(1, s)$-clique-connected nucleus hierarchy (Definition 2.3). This makes it possible to compare how quickly the nucleus hierarchy collapses across domains. Table 14 reports the three hierarchy statistics used here. Figures 8 and 9 visualize the resulting trees and persistence diagrams.

`com-youtube` collapses quickly: from 13,507 branches at $s = 3$ to a single branch at $s = 15$. `com-dblp` keeps the largest branch counts and a constant maximum depth across s, reflecting strong

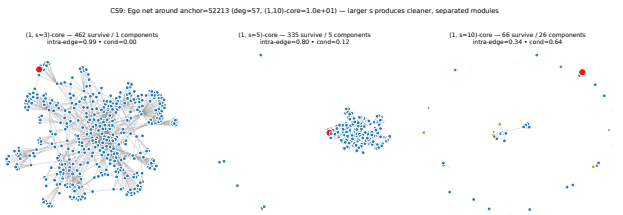

Figure 7: Ego-network around a `com-dblp` anchor under three s values.

Table 14: Nucleus hierarchy summary.

	$s = 3$			$s = 5$			$s = 10$			$s = 15$		
graph	br.	int.	d	br.	int.	d	br.	int.	d	br.	int.	d
com-dblp	10,900	426	4	6,050	278	4	751	20	4	197	8	4
com-youtube	13,507	353	4	714	4	3	12	1	2	1	0	1
web-Stanford	3,455	181	5	1,093	20	3	228	7	3	90	2	2

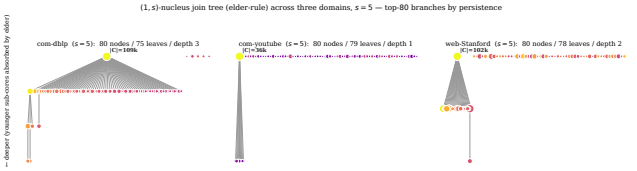

Figure 8: Nucleus join tree at $s = 5$ across three domains.

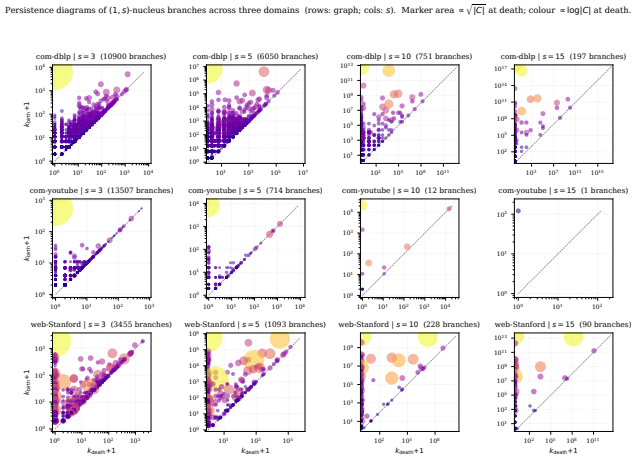

Figure 9: Persistence diagrams across three domains.

nested clique structure. `web-Stanford` retains nontrivial branches at every clique size, including $s = 15$. The hierarchy is therefore not only an output format. It gives a domain-level fingerprint of clique-witness nesting.

Takeaway. These studies support the paper's focus on $(1, s)$. The parameter s changes granularity. On the reported DBLP and YouTube ground-truth tasks, $r = 1$ matches higher-r quality metrics while running much faster. The hierarchy and ego-network studies show that the same vertex-centric decomposition also gives interpretable structure beyond a global ranking. The evidence is not a universal claim about all graph mining tasks. It is a practical justification for optimizing the exact $r = 1$ case.

10 Related Work

The preceding sections specialize a known compact-clique framework to a narrower state model. The related work is best read through that distinction. Prior work explains how to peel cores and nuclei, how to compactly represent cliques, how to update core values locally or in parallel, and how to build hierarchy outputs. This paper uses those ideas, but its main difference is the static-CPI invariant for $r = 1$.

Enumeration-based core and nucleus decomposition. The k-core was introduced by Seidman [12]. Batagelj and Zaveršnik [1] gave the linear-time peeling algorithm with a bucket queue. Nucleus decomposition generalizes this idea from edges to higher-order clique witnesses [11]. In the general (r, s) setting, peeling removes r-cliques according to their support in s-cliques. That generality is broader than the problem solved here. Our work specializes to the vertex-centric $r = 1$ case and exploits a state invariant that does not hold for general r.

Compact clique representations. Bron–Kerbosch search [2], Tomita pivoting [13], and degeneracy ordering [4] are standard tools for clique enumeration. Pivoter introduced the succinct clique tree as a compact representation for clique counting [6]. The Clique Path Index builds on this line to support general nucleus decomposition without explicit s-clique enumeration, including mutating hold/pivot update rules during peeling [9]. Those representations answer how to compactly encode many cliques. This paper asks a narrower question: for vertex peeling, does the implementation need to materialize those path edits? The static-CPI counter form answers no.

Local and parallel peeling. h-index characterizations of core values were studied for local core computation [5, 7, 8]. SPIN adapts that fixed-point view to the implicit s-clique witnesses encoded by the CPI. SPIN* is the peel-order optimization of the same equations. The difference from prior local h-index work is not the operator itself. The difference is that the witness multiset is represented by static CPI paths and updated by binomial support-loss formulas.

Parallel construction and clique counting. Parallel clique search usually tries to distribute the expensive recursive enumeration work. ParaBuild follows the same seed-level decomposition principle. Its specific role in this paper is tied to the static consumer layout. Because the downstream peel never mutates path sets, construction partitions into thread-independent subproblems whose results combine through a deterministic merge. This is an engineering consequence of the $r = 1$ state model rather than a new clique-enumeration paradigm.

Hierarchical cohesive subgraphs. Nucleus decompositions naturally form hierarchies across core levels [11]. Elder-rule join-tree extraction is standard in computational topology [3]. Prior nucleus pipelines can extract such hierarchies after sorting decomposition output or revisiting clique witnesses. BuildHier instead reuses the preserved static CPI: after core values are known, each leaf has a birth level and can be processed as a hyperedge. The hierarchy result is therefore a byproduct of preserving the CPI through peeling.

Empirical graph mining methodology. The evaluation follows common practice in cohesive-subgraph experiments: SNAP community ground truth, large public graphs, runtime and memory comparisons, and dense synthetic stress tests [9, 10, 14]. The case studies use those benchmarks to test whether the $r = 1$ specialization remains useful as a vertex-ranking tool.

11 Conclusion

The paper began with a tension in general CPI-based nucleus decomposition. The index avoids explicit clique enumeration, but the general algorithm keeps it as mutable residual state. For $(1, s)$-nucleus decomposition, that mutability is unnecessary. CPI paths do not need to be mutable residual state. Vertex peeling removes vertices but does not delete edges, so the path structure built at the beginning remains valid throughout the peel. A hold removal kills a path contribution. A pivot removal decreases an effective pivot counter. This changes exact decomposition from residual-index maintenance to counter maintenance over a static symbolic index.

The algorithmic consequences follow from that state change. SPIN states the fixed-point computation over the static CPI. SPIN* realizes the same computation in deletion order. ParaBuild addresses construction after cheap peeling exposes it as the next bottleneck. BuildHier scans static CPI leaves in descending birth level to recover the hierarchy.

The experiments and case studies support the practical value of the specialization. The reported matched configurations agree with the mutable baseline, and the measured pipeline reduces runtime and memory on the evaluated benchmarks. The case studies show that vertex-centric $(1, s)$ can match higher-r quality metrics on the reported community tasks at much lower cost. The boundary is explicit: the static-CPI invariant is for $r = 1$, not for general (r, s).

Several directions extend this work. The most direct is to identify restricted $r \geq 2$ settings where enough path structure survives to support counter-only updates, for example edge peelings constrained to triangle-only support. A second direction is the streaming and dynamic-graph setting: vertex peeling preserves the static CPI, but vertex insertion or deletion changes which paths exist; the question is whether the new paths can be merged into the static index incrementally. A third direction is hardware: SPIN*'s peel reads the static index and writes only path counters, which maps to GPU and other accelerators, while ParaBuild's seed-disjoint structure is a natural fit for distributed-memory partitions. A fourth direction is application-driven: extending the case studies into temporal and attributed graphs, where the static-CPI invariant could anchor a sequence of decompositions across snapshots.

References

[1] Vladimir Batagelj and Matjaž Zaveršnik. 2003. An $O(m)$ Algorithm for Cores Decomposition of Networks. *arXiv preprint cs/0310049* (2003).

[2] Coen Bron and Joep Kerbosch. 1973. Algorithm 457: finding all cliques of an undirected graph. *Commun. ACM* 16, 9 (Sept. 1973), 575–577. https://doi.org/10.1145/362342.362367

[3] Herbert Edelsbrunner and John Harer. 2010. *Computational Topology: An Introduction.* American Mathematical Society.

[4] David Eppstein, Maarten Löffler, and Darren Strash. 2010. Listing All Maximal Cliques in Sparse Graphs in Near-Optimal Time. In *Proc. International Symposium on Algorithms and Computation (ISAAC).* 403–414.

[5] J E Hirsch. 2005. An index to quantify an individual's scientific research output. *Proc Natl Acad Sci U S A* 102, 46 (2005), 16569–16572.

[6] Shweta Jain and C. Seshadhri. 2020. The Power of Pivoting for Exact Clique Counting. In *Proceedings of the 13th International Conference on Web Search and Data Mining* (Houston, TX, USA) *(WSDM '20).* Association for Computing Machinery, New York, NY, USA, 268–276. https://doi.org/10.1145/3336191.3371839

[7] Linyuan Lu, Tao Zhou, Qian-Ming Zhang, and H. Eugene Stanley. 2016. The h-Index of a Network Node and Its Relation to Degree and Coreness. *Nature Communications* 7 (2016), 10168.

[8] Alberto Montresor, Francesco De Pellegrini, and Daniele Miorandi. 2013. Distributed k-core Decomposition. In *IEEE Trans. on Parallel and Distributed Systems*, Vol. 24. 288–300.

[9] Author Placeholder. 2026. Nucleus Decomposition Revisited: An Efficient Counting-Based Approach. In *Proc. ACM SIGMOD International Conference on Management of Data.* Sibling submission; placeholder bibkey to be updated to real citation.

[10] Ahmet Erdem Sariyüce and Ali Pinar. 2018. Peeling Bipartite Networks for Dense Subgraph Discovery. *ACM Trans. Knowl. Discov. Data* 12, 1 (2018).

[11] Ahmet Erdem Sariyüce, C. Seshadhri, Ali Pinar, and Ümit V. Çatalyürek. 2015. Finding the Hierarchy of Dense Subgraphs Using Nucleus Decompositions. In *Proc. International Conference on World Wide Web (WWW).* 927–937.

[12] Stephen B. Seidman. 1983. Network Structure and Minimum Degree. *Social Networks* 5, 3, 269–287.

[13] Etsuji Tomita, Akira Tanaka, and Haruhisa Takahashi. 2006. The Worst-Case Time Complexity for Generating All Maximal Cliques and Computational Experiments. *Theoretical Computer Science* 363, 1 (2006), 28–42.

[14] Jaewon Yang and Jure Leskovec. 2015. Defining and Evaluating Network Communities Based on Ground-Truth. *Knowledge and Information Systems* 42, 1 (2015), 181–213.